Ex 3: Generative Adversarial Networks (GANs) for Synthetic Image Generation

Learning Objective:

Implementationof Generative Adversarial Networks (GANs) for Synthetic Image Generation using the MNIST Dataset.

Steps:

1. Data Preparation

Preprocessing: Load MNIST, scale pixels to $[-1, 1]$ for tanh compatibility, shuffle, and batch the data.

2. The Generator (The Forger)

Architecture: Projects a 100D noise vector into a $7 \times 7 \times 256$ tensor, then upsamples via Conv2DTranspose to reach the final $28 \times 28 \times 1$ image size using tanh.

3. The Discriminator (The Critic)

Architecture: Uses convolutional layers with LeakyReLU and Dropout to downsample the image into a single probability score via a Sigmoid activation.

4. Loss Functions

Objective: Uses Binary Cross-Entropy to penalize the Discriminator for misclassifying images and the Generator for failing to "fool" the critic.

5. Optimizers

Setup: Two separate Adam optimizers are used to independently update the weights of the Generator and Discriminator.

6. Training Step

Execution: Noise is fed to the Generator to create fakes; the Discriminator evaluates both fakes and reals, and gradients are applied to improve both simultaneously.

7. Training Loop

Iteration: The process repeats across epochs, with periodic image generation to monitor the "learning" progress of the Generator.

8. Visualization

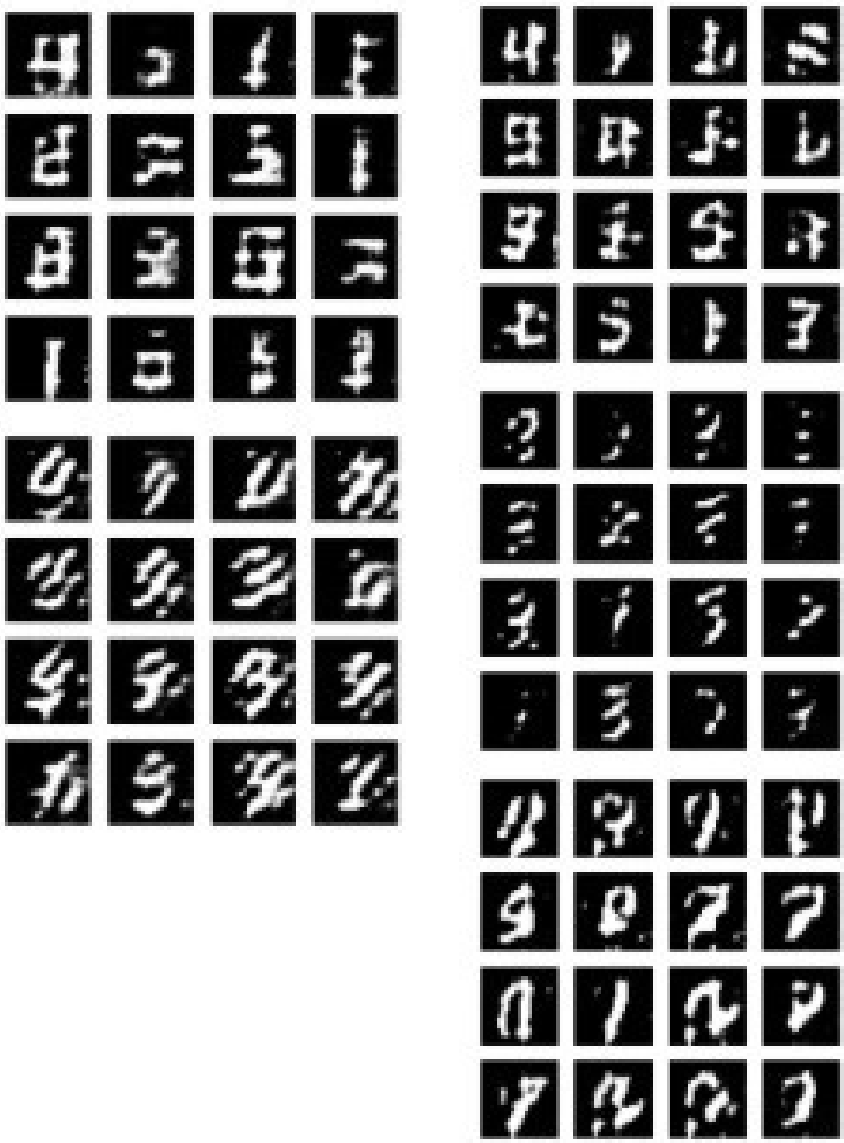
Comparison: Final generated samples are plotted alongside original MNIST digits to evaluate how realistically the model has learned the data distribution.

CODE:

```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = (x_train - 127.5) / 127.5
x_train = x_train.reshape(-1,28,28,1)
BUFFER_SIZE = 60000
BATCH_SIZE = 256
dataset =
tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
def build_generator():



    model = tf.keras.Sequential()
    model.add(layers.Dense(7*7*256, input_shape=(100,)))
    model.add(layers.Reshape((7,7,256)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(128,5,strides=1,padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(64,5,strides=2,padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(1,5,strides=2,padding='same',activation='tanh'))
    return model



def build_discriminator():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64,5,strides=2,padding='same',input_shape=[28,28,1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Conv2D(128,5,strides=2,padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Flatten())
    model.add(layers.Dense(1,activation='sigmoid'))
    return model



generator = build_generator()
```

```
discriminator = build_discriminator() cross_entropy =

tf.keras.losses.BinaryCrossentropy()    g_optimizer    =

tf.keras.optimizers.Adam(0.0002)
d_optimizer = tf.keras.optimizers.Adam(0.0002)
@tf.function
deftrain_step(images):
    noise= tf.random.normal([BATCH_SIZE,100])

    withtf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated_images = generator(noise)

        real_output = discriminator(images)
        fake_output = discriminator(generated_images)
        gen_loss = cross_entropy(tf.ones_like(fake_output), fake_output)

        real_loss = cross_entropy(tf.ones_like(real_output), real_output)
        fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
        disc_loss = real_loss + fake_loss

    gradients_gen = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_disc = disc_tape.gradient(disc_loss, discriminator.trainable_variables)
    g_optimizer.apply_gradients(zip(gradients_gen, generator.trainable_variables))
    d_optimizer.apply_gradients(zip(gradients_disc, discriminator.trainable_variables))

EPOCHS = 5

forepoch in range(EPOCHS):
    forimage_batch in dataset:
        train_step(image_batch)

    noise= tf.random.normal([1,100])
    generated_image = generator(noise, training=False)
    plt.imshow(generated_image[0,:,:,0], cmap='gray')
    plt.title(f"Epoch {epoch+1}")
    plt.show()
```

OUTPUT:



RESULT:

Upon The Completion of this experiment, The GAN model was successfully trained on the MNIST dataset. The generator learned to produce synthetic handwritten digit images by competing against the discriminator.

GAN Lab Experiment – Question and Answer Format

Q1. Identify the section of the code where the MNIST dataset is loaded. Why are the labels ignored in this GAN implementation?

Answer:

The MNIST dataset is loaded in the line:

(x_train, _), (_, _) = keras.datasets.mnist.load_data().

Labels are ignored because GANs do not require labels. The GAN only learns to generate images similar to real images using unsupervised learning.

Q2. Observe the preprocessing step (x_train - 127.5) / 127.5. Why are the pixel values converted to the range [-1, 1]?

Answer:

Pixel values are converted to [-1, 1] because the Generator uses tanh activation in the output layer. This normalization improves training stability.

Q3. Locate the variable noise_dim = 100. What is the purpose of the random noise vector in the Generator?

Answer:

The random noise vector is used as input to the Generator to create synthetic images. Different noise inputs produce different generated images.

Q4. Identify the Generator architecture in the code. How many Dense layers are used in the Generator model?

Answer:

The Generator uses 2 Dense layers followed by a Reshape layer to convert the output into image format.

Q5. Locate the Discriminator model in the program. What is the purpose of the Flatten layer?

Answer:

The Flatten layer converts the 2D image into a 1D vector so that it can be processed by Dense layers for classification.

Q6. Observe the final layer of the Discriminator that uses sigmoid activation. Why is sigmoid used here?

Answer:

Sigmoid is used because the Discriminator performs binary classification (real or fake), and sigmoid outputs values between 0 and 1.

Q7. Identify the loss function used in the GAN. Why is Binary Cross Entropy used in this experiment?

Answer:

Binary Cross Entropy is used because the Discriminator performs binary classification between real and fake images.

Q8. Why is the Discriminator trained using both real images and generated fake images?

Answer:

The Discriminator must learn to distinguish between real and fake images, so it is trained using both types of images.

Q9. In Generator training, fake images are compared with real_labels. Why is this done?

Answer:

This is done to train the Generator to fool the Discriminator into thinking the generated images are real.

Q10. How many synthetic images are displayed at the end of training?

Answer:

16 synthetic images are displayed in a 4×4 grid.

Q11. Change epochs = 20 to epochs = 50 and run the program again. What difference do you observe in the generated images?

Answer:

Increasing epochs improves image quality because the model trains longer and learns better features.

Q12. Change noise_dim from 100 to 50. What happens to the generated images?

Answer:

Reducing noise_dim reduces the variation in generated images and may reduce image quality.

Q13. Increase the number of neurons in the Generator from 128 to 256. Does the output improve?

Answer:

Yes, increasing neurons increases model capacity and can improve image quality.

Q14. Change the batch size from 64 to 128. What impact does this have on training?

Answer:

Increasing batch size makes training more stable but increases memory usage.

Q15. Add one more Dense layer to the Generator. Does the quality of images improve?

Answer:

Yes, adding another layer may improve image quality because the Generator becomes deeper and learns better features.

Q16. Add one more Dense layer to the Discriminator. What change do you observe in training results?

Answer:

The Discriminator becomes stronger and can better classify real and fake images.

Q17. Replace ReLU with LeakyReLU in the networks. Does training become more stable?

Answer:

Yes, LeakyReLU prevents the dying ReLU problem and makes training more stable.

Q18. Observe Generator loss and Discriminator loss during training. What trend do you notice?

Answer:

Generator loss gradually decreases while Discriminator loss fluctuates during training.

Q19. Compare the results of the original GAN and improved GAN. Which one produces clearer digits?

Answer:

The improved GAN produces clearer and more realistic digits.

Q20. Why does the Generator initially produce random noisy images?

Answer:

Because the Generator weights are randomly initialized and it has not learned patterns yet.

Q21. What happens if the Discriminator becomes too strong compared to the Generator?

Answer:

If the Discriminator becomes too strong, the Generator cannot learn and training fails.

Q22. Why is GAN training considered unstable compared to normal neural networks?

Answer:

GAN training is unstable because two networks (Generator and Discriminator) compete with each other during training.

Q23. Mention two real-world applications of GANs other than image generation.

Answer:

Deepfake video generation and data augmentation.

Q24. How can this GAN architecture be modified to generate color images instead of grayscale images?

Answer:

Change the output layer to 3 channels (RGB), increase image size, and modify reshape layer to (height, width, 3).